

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

R.HARIKA(19257
)

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I RASINENI HARIKA(192572140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Harika

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation

Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand
-------------------------	-----	--	-----------------------------------	---------------------------------------	--

1. Sequential File Organization vs Heap File Organization

Feature	Sequential File	Heap File
Insertion	Slow because records must maintain sorted order	Fast , records added wherever space is available
Deletion	Moderate; may require reorganization/marking	Fast; deleted space can be reused
Retrieval	Very efficient for sequential and ordered access	Slow for arbitrary searches; may require full scan
Range queries	Excellent	Poor
Maintenance	Higher	Lower

Sequential file: Records are stored in a particular order, usually according to a key. It is useful for batch processing and range queries.

Heap file: Records are stored in no particular order. It is suitable when insertions are frequent and fast retrieval by key is not the main requirement.

Conclusion: Sequential files are better for **ordered/range retrieval**, while heap files are better for **fast insertion and simple storage**.

2. Clustered Indexing vs Non-Clustered Indexing

Feature	Clustered Index	Non-Clustered Index
Data order	Data records follow index order	Data order is independent
Number generally allowed	Usually one per table	Multiple possible
Range queries	Very efficient	Less efficient
Storage	May require less separate storage	Requires separate index storage
Example	Employee table sorted by Department	Index on EmployeeName

Example

Suppose an employee table is physically stored in Department order.

A **clustered index on Department** keeps CSE employees together, followed by ECE, EEE, etc.

A **non-clustered index on EmployeeName** contains names and pointers to the actual records, while the table itself remains physically ordered by Department.

Conclusion: Clustered indexing is useful for **range and grouped access**, whereas non-clustered indexing is useful when multiple different search attributes are required.

3. Primary Indexing vs Secondary Indexing

Feature	Primary Index	Secondary Index
Search field	Usually primary/ordering key	Non-ordering attribute
Number	Usually one	Multiple possible
Storage	Generally smaller, especially if sparse	Usually larger, often dense
Search	Very fast	Very fast, but may require extra pointer access
Example	StudentID	StudentName

Primary Index

If student records are sorted by StudentID, a primary index can contain one index entry per block.

Secondary Index

An index on StudentName can contain entries for individual records because the file is not physically sorted by name.

Conclusion: Primary indexes generally require **less storage** and provide efficient access to the ordering key. Secondary indexes consume more storage but allow fast searches on additional attributes.

4. Static Hashing vs Dynamic Hashing

Feature	Static Hashing	Dynamic Hashing
Number of buckets	Fixed	Changes dynamically
Growth handling	Poor	Excellent
Overflow	More likely	Reduced
Implementation	Simple	More complex
Scalability	Limited	High

Static Hashing

A fixed hash function maps records to a fixed number of buckets.

Example:

If the database grows beyond expectations, overflow buckets may be required.

Dynamic Hashing

The number of buckets can grow or shrink as records are inserted or deleted. Examples include **extendible hashing** and **linear hashing**.

Conclusion: Static hashing is simple and suitable for stable datasets. Dynamic hashing is better for **growing databases and changing workloads**.

5. B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data pointers	Can occur in internal and leaf nodes	Usually only in leaf nodes
Leaf nodes linked	Usually no	Yes
Range queries	Good	Excellent
Fan-out	Lower	Higher

Feature	B-Tree	B+ Tree
Sequential access	Good	Very efficient
Database usage	Less common	Widely used

B-Tree

Search can terminate at an internal node because data may be stored at different levels.

B+ Tree

All actual record pointers are stored in leaf nodes. The linked leaves allow fast sequential and range access.

For large databases, B+ Trees generally provide:

- Smaller tree height.
- Fewer disk I/Os.
- Efficient range queries.
- Efficient sequential traversal.

Conclusion: B+ Tree is generally preferred for large-scale database indexing, especially when both exact-match and range queries are required.

6. Linear Hashing vs Extendible Hashing

Feature	Linear Hashing	Extendible Hashing
Directory	No large directory required	Uses directory
Bucket splitting	Gradual, in a predetermined sequence	Based on bucket overflow
Growth	Smooth/incremental	Can grow by directory doubling
Memory overhead	Lower	Higher due to directory
Scalability	Excellent	Excellent
Implementation	Relatively simple	More complex

Linear Hashing

Buckets are split progressively as the file grows. It avoids sudden large restructuring.

Extendible Hashing

Uses a directory containing pointers to buckets. When buckets overflow, they split; the directory may double when necessary.

Conclusion: Linear hashing is useful when **smooth growth and lower directory overhead** are desired. Extendible hashing provides very fast bucket access and flexible growth but requires additional directory management.

7. Dense Indexing vs Sparse Indexing

Feature	Dense Index	Sparse Index
Index entries	One per record	One per block/group
Memory usage	High	Low
Search speed	Very fast	Fast, but requires block search
Storage overhead	High	Low
Requirement	Can work with unordered data	Usually requires sorted/ordered data

Dense Index

For 1 million records, the index may contain approximately 1 million entries. Therefore, it provides direct access to records but consumes considerable memory/storage.

Sparse Index

The index may contain only one entry for each data block. After finding the appropriate block, the records inside the block are searched.

Conclusion: Dense indexing gives **faster access but requires more storage**, while sparse indexing saves storage at the cost of slightly slower retrieval.

8. Indexed Sequential File vs Direct File Organization

Feature	Indexed Sequential	Direct/Hashed File
Exact search	Fast	Very fast
Range search	Excellent	Poor
Sequential processing	Excellent	Poor
Insertion	Moderate	Fast
Complexity	Higher	Lower
Index overhead	Required	Usually not required

Indexed Sequential File

Records are stored sequentially according to a key, with an index used for faster access.

Advantages:

- Supports both sequential and direct access.
- Excellent for range queries.
- Suitable for reports and ordered processing.

Limitations:

- Index requires additional storage.
- Insertions/deletions may require overflow areas or reorganization.
- Maintenance is more complex.

Direct File

A hash function directly determines the location of a record.

Advantages:

- Very fast exact-match access.
- Efficient for frequent insertions.

Limitation: Poor for range queries because records are not stored in sorted order.

Conclusion: Indexed sequential organization is more versatile; direct organization is better for **exact-match operations**.

9. Single-Level vs Multi-Level Indexing

Feature	Single-Level Index	Multi-Level Index
Index levels	One	Multiple
Large index handling	Less efficient	More efficient
Search	Requires more index entries to be examined	Fewer disk accesses
Storage	Simple	Additional levels required
Suitable for	Small databases	Large databases

Single-Level Index

A single index directly points to data blocks.

If the index becomes very large, searching the index itself can require many disk accesses.

Multi-Level Index

An index is created on the first-level index.

Structure:

Top-level index → Second-level index → First-level index → Data

This reduces the number of index blocks that must be accessed.

Conclusion: Single-level indexing is simple and suitable for small files. **Multi-level indexing is better for large databases** because it significantly reduces disk I/O.

10. Hashing vs Indexing for Exact-Match and Range Queries

Query Type	Hashing	B+ Tree Indexing
Exact-match	Excellent — $O(1)$ average	Very good — $O(\log N)$
Range query	Poor	Excellent — $O(\log N + k)$
Ordered retrieval	Poor	Excellent
Insert/Delete	Very good on average	Good
Collision/maintenance	Required	Tree balancing required

Exact-Match Queries

Example:

Find Employee where EmployeeID = 1050

Hashing is generally better because the hash function directly identifies the bucket.

Average complexity:

B+ Tree search generally requires:

Range Queries

Example:

Find employees with salary between ₹50,000 and ₹70,000

A **B+ Tree** is much better because its leaf nodes are ordered and linked.

Approximate complexity:

where k is the number of records returned.

Hashing cannot efficiently locate a continuous range because similar values may be distributed across different buckets.

Final conclusion

- **Exact-match queries → Hashing**
- **Range queries → B+ Tree indexing**
- **Mixed workloads → B+ Tree or use both structures where justified**

Thus, the choice depends mainly on the type of queries performed by the database.